

Wolfie Delivery - Product Specification for TestSprite

Project Overview

Wolfie Delivery is a production-focused food delivery platform targeting New York City.

Architecture:

- Frontend: React + TypeScript + Vite on Vercel
- Backend: Flask on Render
- Database: Supabase PostgreSQL
- Realtime: Flask-SocketIO + Redis
- Maps: Mapbox
- Payments: Stripe (currently test/mock)

Main Applications

1. Customer App
2. Restaurant Dashboard
3. Driver App
4. Admin Dashboard

Core Order Lifecycle

Customer creates order -> Backend validates -> Saved to Supabase ->
Restaurant receives realtime notification -> Restaurant accepts ->
Driver assigned -> Driver accepts -> Restaurant marks Ready ->
Driver picks up -> Delivered -> Customer receives realtime updates.

Testing Objectives

Verify authentication, authorization, API correctness, realtime events, Redis, database consistency, UI behaviour, error handling, websocket stability, performance, deployment on Vercel/Render, and production readiness.

Instructions to TestSprite

You are NOT only a testing agent.

Act simultaneously as:

- Senior QA Engineer
- Senior Software Architect
- Senior Product Reviewer
- Senior DevOps Reviewer
- Security Reviewer

Your responsibilities:

- Explore the application like a real user.
- Generate and execute an intelligent test plan.
- Perform end-to-end testing across Customer, Restaurant, Driver and Admin.
- Verify UI, API, database, Redis and WebSocket consistency.
- Investigate every failure until the root cause is identified.
- Include screenshots, console logs, network logs and severity.

Additionally review the PRODUCT itself:

- Suggest missing features compared to Uber Eats, DoorDash and Grubhub.

- Review order lifecycle and recommend improvements.
- Review restaurant workflow.
- Review driver workflow.
- Review admin workflow.
- Review customer UX.
- Review scalability.
- Review security.
- Review database design.
- Review API design.
- Review deployment architecture.
- Review monitoring and observability.
- Suggest AI features and operational improvements.

Do not assume the current implementation is correct.
Challenge design decisions.
Recommend better architectures where appropriate.

Final report must include:

- Overall score (/100)
- Production readiness
- Critical / High / Medium / Low issues
- Business recommendations
- Technical recommendations
- Architecture recommendations
- Prioritized roadmap before launch.